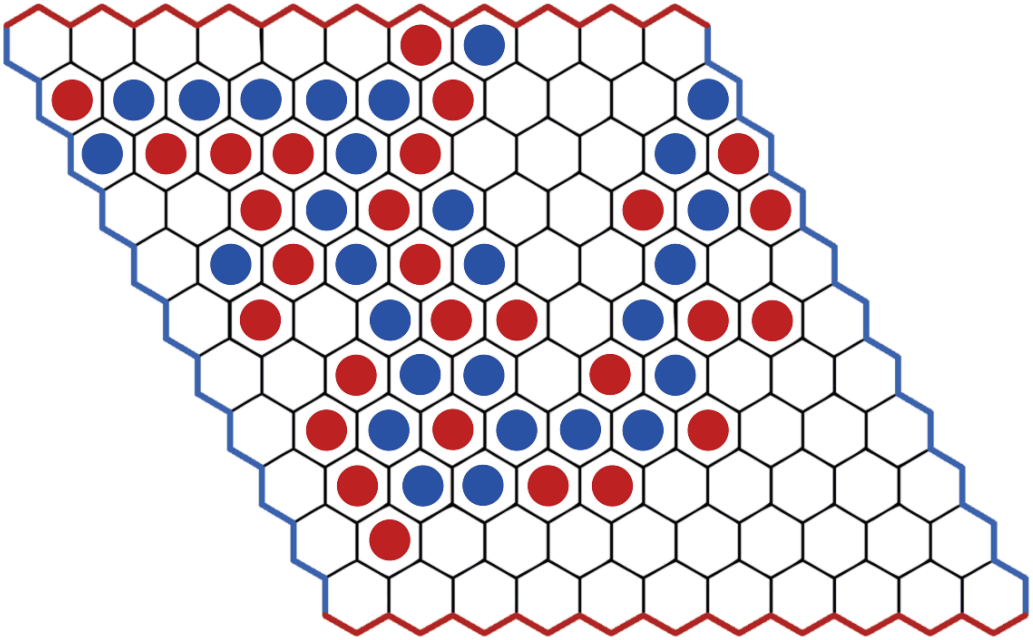
HEXABEL

Agente inteligente para jugar Hex utilizando técnicas de AI



Autor: Abel de la Noval Pérez

Algoritmo: Minimax + Poda Alfa-Beta + Iterative Deepening

Tiempo límite: 4.8 segundos

Proyecto: Inteligencia Artificial

Índice

1. Introducción **3**

2. Arquitectura del sistema **3**

3. Representación del Tablero y Topología **4**

4. Flujo de ejecución del agente **5**

4.1. Inicio del turno 5

4.2. Chequeo de jugadas críticas 6

4.3. Generación de movimientos candidatos 6

4.4. Ordenación de movimientos 6

4.5. Evaluación de movimientos mediante Minimax 7

4.6. Selección final de la jugada 7

5. Algoritmo principal **8**

5.1. Minimax 8

5.2. Poda Alfa-Beta 8

5.3. Profundización Iterativa (Iterative Deepening) 9

6. Optimizaciones **9**

6.1. Killer Moves 9

6.2. Late Move Reduction (LMR) 10
6.3. Tablas de transposición (Transposition Tables) 10

7. Heurística y Evaluación **11**
7.1. Caminos primarios y secundarios 11
7.2. Cálculo del coste de celdas 11
7.3. Evaluación combinada 12

8. Conclusiones **12**

Introducción

En este proyecto se desarrolló un agente inteligente para jugar Hex, un juego de tablero de dos jugadores en el que cada uno busca conectar lados opuestos mediante fichas en un tablero hexagonal. El objetivo del agente es seleccionar, en cada turno, el movimiento que maximice la probabilidad de ganar, considerando tanto la posición actual como posibles respuestas del rival.

Para lograrlo, se combinaron varias técnicas de inteligencia artificial: se utiliza Minimax para explorar el árbol de juego, poda alfa-beta para reducir el número de nodos evaluados, Iterative Deepening para garantizar siempre una jugada disponible en el tiempo asignado, tablas de transposición (TT) para reutilizar evaluaciones previas, y técnicas de optimización como killer moves y Late Move Reduction (LMR). Estas herramientas permiten al agente tomar decisiones estratégicas de manera eficiente y robusta.

Arquitectura del sistema

La arquitectura del sistema se ha diseñado siguiendo un modelo modular, que permite separar claramente las responsabilidades de cada componente y facilita futuras mejoras o modificaciones.

El proyecto se organiza principalmente en los siguientes módulos:

- **board.py**: Contiene la clase `HexBoard`, encargada de representar el tablero del juego. Sus responsabilidades incluyen:
    - Inicialización del tablero de tamaño $n \times n$.

- Gestión de las celdas ocupadas y vacías.
- Verificación de la existencia de una conexión ganadora para un jugador.
- Funciones de utilidad como copia del tablero para simulaciones.

- **player.py**: Define la clase genérica `Player`, que representa un jugador en el juego. Esta clase establece la interfaz básica para cualquier tipo de jugador, incluyendo métodos como `play()`.
- **smart_player.py**: Implementa la clase `SmartPlayer`, que hereda de `Player`. Esta clase encapsula toda la lógica de la Inteligencia Artificial:
  - Algoritmo de búsqueda Minimax con poda Alpha-Beta.
  - Profundización iterativa para garantizar respuestas dentro de límites de tiempo.
  - Heurística avanzada para evaluar posiciones no terminales.
  - Optimización mediante *Transposition Tables*, *Killer Moves*, *Late Move Reduction* y selección de movimientos relevantes.
- **testgame.py**: Script de simulación que permite ejecutar partidas completas entre jugadores, probando la IA y verificando su comportamiento en distintos escenarios.

Representación del Tablero y Topología

El tablero del juego Hex se representa como una matriz bidimensional de tamaño $N \times N$. Cada celda de la matriz corresponde a una casilla hexagonal del tablero real.

Para modelar la conectividad hexagonal, cada celda se considera adyacente a un máximo de seis vecinos. Esta relación de vecindad se implementa mediante un conjunto fijo de desplazamientos:

$$(-1,0),(-1,1),(0,-1),(0,1),(1,-1),(1,0)$$

Este enfoque permite embeber la estructura hexagonal dentro de una matriz cuadrada, manteniendo las seis direcciones de conexión necesarias según las reglas del juego.

A diferencia de otras implementaciones basadas en filas pares e impares (offset coordinates), en este proyecto se utiliza una topología de vecinos fija. Esta decisión proporciona una estructura de grafo uniforme en todo el tablero, lo que simplifica la implementación de los algoritmos de búsqueda y facilita el cálculo de heurísticas basadas en conectividad y distancia.

Flujo de ejecución del agente

El flujo de ejecución del agente se organiza en una secuencia de pasos que garantizan decisiones estratégicas en cada turno de Hex. A continuación se describe cada etapa, mencionando los algoritmos y técnicas utilizadas de forma resumida y comprensible.

Inicio del turno

Al inicio de cada turno, el agente recibe el tablero actual (`HexBoard`) con las posiciones de ambas fichas. En este momento, se inicializan las estructuras internas esenciales para la evaluación de los movimientos, incluyendo la tabla de transposición (TT), los *killer moves* y los temporizadores que controlan el tiempo máximo asignado para cada turno. Estas inicializaciones aseguran que la búsqueda y la evaluación de los movimientos posteriores se realicen

dentro de los límites de tiempo y con la información disponible más reciente.

Chequeo de jugadas críticas

El agente comienza evaluando si existe una victoria inmediata disponible para sí mismo. En caso afirmativo, se selecciona esa jugada de manera prioritaria, garantizando la obtención de la victoria cuando sea posible. Si no existe una victoria inmediata, se verifica si es necesario bloquear una victoria inminente del rival. Ambas jugadas, ya sean ofensivas o defensivas, se priorizan por encima de cualquier otra evaluación, asegurando que las oportunidades decisivas o las amenazas críticas se gestionen antes de considerar estrategias más complejas.

Generación de movimientos candidatos

Una vez evaluadas las jugadas críticas, se generan los movimientos candidatos que el agente puede considerar. La selección se centra en las celdas adyacentes a piezas ya colocadas, siguiendo la estrategia conocida como "active zone", que reduce el espacio de búsqueda a posiciones estratégicamente relevantes. En situaciones donde no se identifican celdas relevantes, se consideran todas las posiciones vacías del tablero, garantizando que el agente siempre tenga alternativas válidas para evaluar.

Ordenación de movimientos

Los movimientos candidatos se ordenan mediante heurísticas ligeras que priorizan aquellos que parecen más prometedores. Entre los criterios considerados se incluyen la proximidad

al centro del tablero y la presencia de vecinos aliados. Esta ordenación no determina el valor final de los movimientos, pero permite que los más prometedores se evalúen primero durante la búsqueda profunda mediante `Minimax`, incrementando la eficiencia y la probabilidad de poda temprana de ramas irrelevantes.

Evaluación de movimientos mediante Minimax

A continuación, el agente evalúa los movimientos mediante el algoritmo `Minimax`, que simula todos los posibles escenarios futuros alternando los turnos del agente y del rival. La búsqueda incorpora técnicas de optimización, incluyendo poda alfa-beta para cortar ramas irrelevantes, *killer moves* para explorar primero aquellos movimientos que generaron poda previamente, y *Late Move Reduction (LMR)* para reducir la profundidad de evaluación de movimientos aislados o poco prometedores. La búsqueda se realiza mediante *Iterative Deepening*, evaluando primero a menor profundidad y aumentando progresivamente, de manera que siempre exista una mejor jugada conocida dentro del límite de tiempo.

Selección final de la jugada

Finalmente, el agente selecciona la jugada con el mayor valor estimado. En caso de detectar una victoria inmediata (valor heurístico ≥ 9000), esta jugada se prioriza automáticamente sobre cualquier otra evaluación. Este procedimiento garantiza que el agente combine decisiones estratégicas a largo plazo con la gestión de jugadas críticas inmediatas, ofreciendo un comportamiento robusto y eficiente durante todo el juego.

Algoritmo principal

La toma de decisiones del agente se basa en un algoritmo de búsqueda que combina el método `Minimax` con optimizaciones específicas para mejorar la eficiencia y garantizar decisiones estratégicas precisas dentro del límite de tiempo.

Minimax

El algoritmo `Minimax` es la base de la evaluación de movimientos. Su objetivo es simular los posibles escenarios futuros alternando los turnos del agente (maximizador) y del rival (minimizador). Para cada movimiento candidato, el algoritmo evalúa recursivamente los posibles estados resultantes, asignando un valor heurístico que refleja la ventaja relativa del agente. Las decisiones se toman siempre considerando que el rival actuará de manera óptima para minimizar la ganancia del agente. Esta evaluación permite que el agente no solo contemple la situación inmediata, sino que anticipe consecuencias a varios turnos de distancia, construyendo un razonamiento estratégico robusto.

Poda Alfa-Beta

Para mejorar la eficiencia de la búsqueda, `Minimax` se combina con la técnica de **poda alfa-beta**. Este método permite cortar ramas del árbol de decisión que no pueden mejorar el resultado actual. Mientras se recorre el árbol, se mantienen dos valores: *alfa*, el valor mínimo garantizado para el jugador maximizador, y *beta*, el valor máximo garantizado para el jugador minimizador. Si en algún punto se detecta que una rama no puede superar estos

límites, se interrumpe su evaluación y se procede al siguiente movimiento. Esta técnica reduce significativamente el número de nodos evaluados, permitiendo al agente explorar movimientos más profundos dentro del tiempo disponible.

Profundización Iterativa (Iterative Deepening)

Dado que el tiempo de cálculo por turno es limitado, la búsqueda se realiza mediante **profundización iterativa**. Este enfoque consiste en evaluar primero los movimientos a baja profundidad y luego ir aumentando progresivamente la profundidad de análisis, manteniendo siempre disponible la mejor jugada conocida hasta el momento.

Optimizaciones

Para mejorar la eficiencia y robustez del algoritmo principal, se incorporan varias técnicas de optimización que permiten reducir el número de nodos evaluados y enfocar la búsqueda en los movimientos más prometedores.

Killer Moves

Los *killer moves* son movimientos que en iteraciones previas del árbol de búsqueda causaron poda alfa-beta en la misma profundidad. El algoritmo prioriza estos movimientos al inicio de la lista de candidatos, aumentando la probabilidad de poda temprana y reduciendo

la cantidad de nodos innecesarios que se evalúan. Esta estrategia permite al agente explorar primero los movimientos más prometedores basados en experiencia pasada, mejorando significativamente la eficiencia sin afectar la calidad de la decisión final.

Late Move Reduction (LMR)

El *Late Move Reduction* consiste en reducir temporalmente la profundidad de evaluación de movimientos considerados poco prometedores, típicamente aquellos aislados o con pocas conexiones estratégicas. Si un movimiento reducido demuestra ser más prometedor de lo esperado, se reevalúa a profundidad completa, asegurando que no se pierda ningún movimiento crítico. Esta técnica permite ahorrar tiempo de cálculo en movimientos de baja relevancia, enfocando recursos en las jugadas que realmente afectan el resultado del juego.

Tablas de transposición (Transposition Tables)

Las tablas de transposición almacenan los resultados de posiciones previamente evaluadas, permitiendo que el algoritmo reutilice estos valores en futuras llamadas si se alcanza la misma configuración del tablero. Cada entrada guarda la profundidad a la que se evaluó la posición, el tipo de valor (exacto, límite superior o límite inferior) y el valor heurístico calculado. Este mecanismo evita la recalculación de nodos repetidos, aumentando la eficiencia del agente y reduciendo el tiempo total de búsqueda, especialmente en tableros grandes con múltiples caminos similares.

Heurística y Evaluación

La heurística del agente permite asignar un valor numérico a cada posición del tablero, proporcionando un criterio para que el algoritmo principal (`Minimax`) compare y seleccione la mejor jugada. Esta evaluación combina múltiples factores estratégicos para reflejar tanto la ventaja inmediata como la fortaleza a largo plazo de la posición.

Caminos primarios y secundarios

Para evaluar la conectividad del agente y del rival, se calculan dos tipos de caminos. El **camino primario** representa la ruta más corta que conecta los lados opuestos del tablero, mientras que el **camino secundario** proporciona alternativas de respaldo, evitando las celdas que forman parte del camino primario. Esta distinción permite al agente considerar no solo el progreso directo hacia la victoria, sino también la robustez de su posición frente a posibles bloqueos del adversario.

Cálculo del coste de celdas

Cada celda del tablero recibe un coste según su ocupación y relevancia estratégica. Las celdas ocupadas por el agente tienen coste mínimo, ya que representan conexiones ya establecidas, mientras que las celdas vacías tienen un coste bajo pero positivo, reflejando la necesidad de colocarlas para completar un camino. Las celdas ocupadas por el rival reciben un coste alto, lo que refleja que estas posiciones bloquean los caminos y dificultan la conectividad. En el caso de evitar temporalmente ciertas celdas estratégicas (como en el cálculo

de caminos secundarios), se les asigna un coste elevado pero pasable, permitiendo que el algoritmo considere alternativas si es necesario.

Evaluación combinada

El valor final de la heurística se calcula combinando los costes de los caminos primario y secundario tanto del agente como del rival. Esta combinación pondera la importancia del camino principal y la robustez que proporciona el camino secundario, resultando en un único valor heurístico que refleja la ventaja relativa de la posición. De esta forma, el agente puede identificar no solo los movimientos que generan progreso inmediato hacia la victoria, sino también aquellos que fortalecen su posición estratégica frente a bloqueos futuros.

Conclusiones

El desarrollo del agente para el juego de Hex ha permitido combinar técnicas clásicas de inteligencia artificial con estrategias de optimización modernas, resultando en un sistema capaz de tomar decisiones estratégicas eficientes y robustas. La integración de `Minimax` con poda alfa-beta y profundización iterativa asegura que el agente pueda anticipar los movimientos del rival y planificar sus jugadas considerando múltiples escenarios futuros.

Las optimizaciones implementadas, incluyendo *killer moves*, *Late Move Reduction* y tablas de transposición, contribuyen a reducir el tiempo de cálculo y a enfocar la búsqueda en los movimientos más prometedores, manteniendo la calidad de la decisión incluso en tableros grandes o complejos. La heurística avanzada basada en caminos primarios y secundarios permite evaluar la robustez de la posición del agente frente a bloqueos y amenazas del rival, garantizando una estrategia sólida a corto y largo plazo.

En conjunto, el proyecto demuestra cómo la combinación de algoritmos de búsqueda, técnicas de optimización y heurísticas estratégicas permite construir un agente competitivo capaz de gestionar situaciones complejas de manera eficiente.